Ders 26 Genişletilmiş Notu: Sıralama Algoritmalarının Karşılaştırılması, Counting Sort ve Radix Sort

Ders 26 Genişletilmiş Notu: Sıralama Algoritmalarının Karşılaştırılması, Counting Sort ve Radix Sort

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dönem Projesi Üzerine Soru-Cevap: Kurulum mu, Arama mı?
- 2. Sıralama Algoritmalarının İki Ailesi
- 3. Karşılaştırmaya Dayalı Algoritmaların Karmaşıklık Tablosu
- 4. Diğer Karşılaştırma Ölçütleri: Kararlılık, Yerinde Sıralama, Online ve Offline
- 5. Algoritmaların Kullandığı Stratejiler
- 6. Hangi Durumda Hangi Algoritma?
- 7. Sayma Sıralaması (Counting Sort)
- 8. Taban Sıralaması (Radix Sort)
- 9. Radix Sort'un Karmaşıklığı ve $n \log n$ Sınırı
- 10. Dizgilerde (String) Radix Sort
- 11. LSD ve MSD Radix Sort
- Örnek Sorular
- Dersin Toparlaması

1. Dönem Projesi Üzerine Soru-Cevap: Kurulum mu, Arama mı?

Bu ders dönemin son dersiydi. Hoca bugün sıralama algoritmalarını toplu olarak karşılaştıracağını ve birkaç yeni sıralama tekniği göstereceğini söyledi. Konuya geçmeden önce dönem projesiyle ilgili bir soru geldi. Sorunun akademik yanı, veri yapısı tasarımında sık karşılaşılan bir ödünleşimdir (trade-off).

Soru: Projede yer ve zaman optimizasyonu isteniyor. Bu optimizasyon veri yapısı hazırlanırken (doldurulurken) mi, yoksa kullanılırken mi bekleniyor? İki çözüm düşünülebilir: birincisinde veri yapısını kurmak kolay ama içinde arama yapmak yavaş; ikincisinde veri yapısını kurmak biraz daha uzun sürüyor ama ortaya çıkan yapıda arama yapmak hızlı. İkincisi seçilebilir mi?

Cevap: Bu, veri yapısının ne sıklıkla oluşturulduğuna bağlıdır.

- Yapı bir kez oluşturulup sonra tekrar tekrar aranacaksa, aramanın verimli olması daha mantıklıdır. Kurulumdaki ek maliyet bir kez ödenir.
- Yapı sık sık yeniden oluşturuluyorsa, verimlilik yükünü kurulum aşamasına vermek gerekir.

Problem tanımı bu sıklığı söylemiyorsa, varsayımı açıkça yazmak gerekir. Örneğin "yapının bir kez oluşturulduğunu ve tekrar tekrar arama yapıldığını varsaydım, bu yüzden optimizasyonu arama üzerinden yaptım" denebilir. Hocanın beklentisi, yapılan seçimlerin argümanlarla desteklenmesidir.

Hoca optimizasyon beklentisini de açtı. Akla ilk gelen kaba kuvvet (brute force) çözüm beklenmiyor. Çözümde iyileştirilebilecek adımlar varsa bunların çalışmaya yansıması bekleniyor. Öte yandan yüzde yüz en mükemmel çözüm de şart koşulmuyor. Hocanın benzetmesiyle, "bu da çalışıyor" denilerek kulağı tersten göstermek gibi dolambaçlı çözümler istenmiyor.

2. Sıralama Algoritmalarının İki Ailesi

Sıralama algoritmaları birinci sınıftan beri görülüyor. Bunun nedeni, sıralı veri üzerinde çalışmanın birçok problemin çözümünü kolaylaştırmasıdır. Bu yüzden veri çoğu zaman önce sıralanır, sonra sıralı veri üzerinden yeni çözümler üretilir.

Hoca sıralama algoritmalarını iki gruba ayırdı:

- **Karşılaştırmaya dayalı (comparison-based) algoritmalar:**
  - Birinci sınıfta öğrenilen temel algoritmalar: bubble sort, selection sort, insertion sort.
  - $O(n \log n)$ karmaşıklığa sahip olanlar: merge sort, quicksort, heap sort.
- **Karşılaştırma yapmayan (non-comparison) algoritmalar:**
  - Sayma sıralaması (counting sort).
  - Taban sıralaması (radix sort). İki türü vardır: en düşük anlamlı basamaktan başlayan (least significant digit, LSD) ve en yüksek anlamlı basamaktan başlayan (most significant digit, MSD).
  - Hoca bu gruba kova sıralamasının (bucket sort) da eklenebileceğini söyledi.

Hoca, $O(n \log n)$ algoritmaların karşılaştırmaya dayalı grubun alt sınırını oluşturduğunu vurguladı. Karşılaştırma yaparak sıralayan bir algoritmayı bundan daha düşük karmaşıklıkla tasarlamak mümkün değildir. Çok sayıda veri sıralanacaksa genelde merge sort, quicksort ve heap sort'tan biri tercih edilir.

> **Ek bilgi (derste anlatılmadı):** Bu alt sınır karar ağacı (decision tree) modeliyle gösterilir. n elemanın $n!$ farklı sıralaması vardır ve her karşılaştırma iki sonuç verir. Bu yüzden her girdiyi doğru sıralayan bir algoritmanın karar ağacının yüksekliği en az $\log_2(n!)$ olmalıdır. $\log_2(n!) = \Theta(n \log n)$ olduğundan en kötü durumda $\Omega(n \log n)$ karşılaştırma gerekir.

Karşılaştırma ile kastedilen şudur: yan yana duran elemanlar ya da dizinin geri kalan elemanları birbiriyle karşılaştırılır ve mutlaka büyük-küçük durumu kontrol edilir. Karşılaştırma yapmayan yöntemlerde ise başka bir yaklaşım uygulanır. Bu yöntemler $n \log n$ karmaşıklığı $O(n)$ düzeyine indirmeye yardım eder, ama bunun için ek alan kullanırlar.

Hoca karşılaştırmaya dayalı algoritmaları veriyi nasıl hareket ettirdiklerine göre de ayırdı:

- Bubble sort ve selection sort yer değiştirme (swapping) ile çalışır.
- Insertion sort kaydırma (shifting) özelliğiyle öne çıkar.
- Quicksort ve heap sort yine yer değiştirme ile çalışır.
- Merge sort ek bellek (extra memory) kullanır ve veriyi kopyalar.

Quicksort ve merge sort, gelecek dönem Algoritma Analizi dersinde böl ve yönet (divide and conquer) algoritmaları altında ayrıntılı işlenecek.

3. Karşılaştırmaya Dayalı Algoritmaların Karmaşıklık Tablosu

Hoca yeni bir sayfa açıp algoritmaları zaman karmaşıklığına (time complexity) ve bellek karmaşıklığına (space complexity) göre bir tabloya yazdı. Zaman için en iyi (best case), ortalama (average case) ve en kötü durum (worst case), bellek için en kötü durum verildi.

Algoritma	En iyi	Ortalama	En kötü	Bellek (en kötü)
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

Satırlarla ilgili hocanın açıklamaları şunlardır:

- **Bubble sort:** En iyi durumun $O(n)$ olabilmesi için iyileştirilmiş (enhanced) sürüm gerekir. İki `for` döngüsüyle yazılan sürümde bu olmaz. Dış döngü, bir turda yer değiştirme olup olmadığını kontrol eden bir `while` döngüsü olmalıdır.
- **Selection sort:** $O(n^2)$'den başka çıkış yoktur. Girdi sıralı olsa bile her adımda kalan elemanların en küçüğü aranır.
- **Insertion sort:** En iyi durumunun $O(n)$ olması önemli bir artıdır. Birçok küçük veri setinde bu yüzden insertion sort tercih edilir.
- **Quicksort:** Çok tercih edilen bir algoritmadır, ama en kötü durumu $O(n^2)$'dir. Hoca, tersten sıralıya yakın bir veri setinde quicksort'un tercih edilmemesini söyledi.

  Ders sonrası not: Hoca bellek sütununu sözlü okurken merge sort ve quicksort için de $O(1)$ dedi. Doğrusu: merge sort birleştirme için $O(n)$ ek dizi kullanır. Quicksort'un ek belleği özyineleme yığınından (recursion stack) gelir; ortalamada $O(\log n)$, en kötü durumda $O(n)$'dir. Hocanın biraz sonra merge sort'u yerinde olmayan algoritmalar arasında sayması da bu düzeltmeyle uyumludur.

  Ders sonrası not: Quicksort'un en kötü durumu yalnız tersten sıralı veriye özgü değildir, pivot seçimine bağlıdır. İlk ya da son elemanı pivot alan sürümde zaten sıralı bir dizi de en kötü durumu üretir, çünkü her bölmede bir taraf boş kalır.

4. Diğer Karşılaştırma Ölçütleri: Kararlılık, Yerinde Sıralama, Online ve Offline

Hoca zaman ve bellek karmaşıklığının birinci öncelik olduğunu, ama başka ölçütlerin de olduğunu söyledi ve öğrencilere sordu. Tahtaya üç ölçüt yazıldı: kararlı ve kararsız (stable vs. unstable), yerinde (in-place), çevrim içi ve çevrim dışı (online vs. offline). Hoca önce "instable" yazdı, sonra doğrusunun "unstable" olduğunu söyleyip düzeltti.

4.1 Kararlılık (Stability)

Soru: Bir algoritmanın stable olması ne demektir? Bir öğrenci "tutarlı" diyebilir miyiz diye sordu. Hoca, Türkçe karşılık olarak bu olabilir ama asıl önemli olanın ne anlama geldiği olduğunu söyledi ve tahtadaki örnek üzerinden yorumlanmasını istedi.

Tahtadaki dizi sıralandığında `1 2 3 3 4 5 5` elde edilir. Dizide iki tane 3 ve iki tane 5 vardır.

Kararlı (stable) sıralama: Aynı değere sahip iki veri varsa, sıralama bu verilerin sırasını bozmadan yapılır. Girdide ilk görülen 3 çıktıda da ilk sıradaysa ve ikinci görülen 3 ikinci sıradaysa, 5'ler için de aynısı geçerliyse, algoritma kararlıdır.

Kararsız (unstable) sıralama: Algoritma bu konuda bir garanti vermiyorsa kararsızdır. Burada önemli olan "garanti" kelimesidir. Kararsız bir algoritmada ilk 5'in ikinci 5'ten sonra gelmesi her zaman olmak zorunda değildir, ama olabilme ihtimali vardır. Algoritma kesinlik sunmuyorsa ona unstable denir.

Hoca iki listeyi tahtaya yazdı:

- **Kararlı algoritmalar:** bubble sort, insertion sort, merge sort, counting sort, radix sort.
- **Kararsız algoritmalar:** selection sort, quicksort, heap sort.

$O(n \log n)$ karmaşıklığa sahip algoritmalar arasında kararlı olan yalnız merge sort'tur. Bu yüzden hem büyük veri hem kararlılık gerekiyorsa merge sort öne çıkar.

Peki ilk 3'ün önce, ikinci 3'ün sonra gelmesi gerçekten bu kadar önemli mi? Hocanın cevabı: bazı problemler için evet. Aynı değere sahip verilerin veri setindeki sırası anlam taşıyorsa, sıralamada bunun gözetilmesi gerekir. Ama her problemde böyle bir zorunluluk yoktur.

4.2 Yerinde Sıralama (In-Place)

Yerinde (in-place) sıralama algoritması, herhangi bir ek bellek kullanmadan sıralama işlemini gerçekleştiren algoritmadır. Hoca, karmaşıklık tablosundaki bellek sütununa bakılarak bulunabileceğini söyledi:

- Yerinde olanlar: bubble sort, selection sort, insertion sort, quicksort, heap sort. Bu algoritmalar sıralamayı dizinin kendi üzerinde yapar.
- Yerinde olmayanlar: merge sort ve counting sort. Bu iki algoritma ek alan kullanma özelliğiyle öne çıkar.

Ek alan kullanımı bazı ortamlarda dezavantajdır. Gömülü sistemler (embedded systems) gibi belleğin kısıtlı olduğu yerlerde yerinde sıralama algoritmaları tercih edilir.

4.3 Online ve Offline Sıralama

Hoca öğrencilerden tahminde bulunmalarını istedi, sonra açıkladı:

- **Online sıralama algoritması:** Sıralama sürerken dışarıdan ek bir eleman geldiğinde bu durumu idare edebilir (handle). Yeni eleman, sıralamayı bozmadan algoritmanın içine eklenip olması gereken yere yerleşebilir.
- **Offline sıralama algoritması:** Yeni gelen elemanı idare edemez. Sıralama belirli bir kural bütünüyle ilerlediği için, sonradan gelen elemana uyum sağlama (adaptasyon) yeteneği düşüktür.

Algoritmaların çoğu offline'dır. Hoca hangisinin online olduğunu sordu; cevap gelmeyince kendisi yazdı: **insertion sort**. Insertion sort'ta dizi yarıya kadar sıralanmış da olsa, hiç sıralanmamış da olsa, sıralama bitmek üzere de olsa, dışarıdan gelen eleman kaydırma (shift) yeteneği sayesinde olması gereken yere yerleşir. Diğer algoritmalar offline'dır.

5. Algoritmaların Kullandığı Stratejiler

Hoca algoritmaların hangi stratejiyle çalıştığını da sıraladı:

- **Böl ve yönet (divide and conquer):** merge sort, quicksort. Gelecek yıl ayrıntılı görülecek.
- **Heap yapısı:** heap sort, bir veri yapısı olan heap'i kullanır.
- **"Hashing":** Hoca bu başlık altında counting sort'u yazdı.
- **İç içe döngü (nested loop):** klasik algoritmalar, selection sort ve insertion sort.

  Ders sonrası not: Hoca counting sort'u bu listede karşılaştırmaya dayalı algoritmaların stratejileri arasında "hashing" olarak andı. Counting sort karşılaştırma yapmaz ve bir özetleme (hash) fonksiyonu kullanmaz. Elemanın değerini doğrudan sayaç dizisinin indisi olarak kullanır (anahtar indeksli sayma, key-indexed counting).

Selection sort ve insertion sort'u diğer iç içe döngülü algoritmalardan ayıran şey, artımlı (incremental) bir yaklaşım kullanmalarıdır:

- **Selection sort** her adımda dizinin sıradaki en küçük elemanını bulur: önce en küçüğü, sonra en küçük ikinciyi, sonra en küçük üçüncüyü bularak ilerler.
- **Insertion sort** her adımda bir elemanı olması gereken yere yerleştirir. Elemanların sıralanmasını adım adım iyileştirir ve son elemanı da yerine koyduğunda sıralama tamamlanır.

6. Hangi Durumda Hangi Algoritma?

Hoca genel söylenenlere ek olarak, hangi durumda hangi algoritmanın seçileceğini maddeler hâlinde yazdı:

Durum	Önerilen algoritma	Gerekçe
Küçük veri seti (small input)	Insertion sort	Kararlı, yerinde, en iyi durumda $O(n)$
Büyük veri seti (larger dataset)	Quicksort, merge sort, heap sort	$O(n \log n)$
Neredeyse sıralı veri	Insertion sort	En iyi duruma yakın çalışır
Bellek kullanımı önemli	Heap sort, ardından quicksort	Heap sort $O(1)$ ek bellekle çalışır

Durum	Önerilen algoritma	Gerekçe
Bağlı liste (linked list) sıralama	Merge sort	
Paralelleştirme imkânı var	Merge sort	
Kararlılık gerekli	Quicksort ve heap sort elenir	İkisi de kararsızdır

6.1 Önbellek Dostu (Cache-Friendly) Algoritma

Hoca quicksort'un önbellek dostu (cache friendly) bir algoritma olduğunu söyledi ve bunun ne demek olduğunu sordu.

Soru: Bir algoritmanın önbelleği iyi kullanması neden önemlidir?

Cevap (hoca): Veriler bellekte saklanır ve ana belleğe (primary memory) erişim zaman alır. Belirli veri bütünleri bellekten önbelleğe (cache) alınır ve işlem önbellek üzerinden yapılır. İşlemci tarafındaki L1, L2, L3 önbelleklerinin varlık nedeni budur: her veriyi tek tek gidip RAM'den çekmek yerine, veriyi blok hâlinde alıp önbelleğe koymak ve oradan işlemek performansı büyük ölçüde artırır. Bu bloklardan iyi yararlanan algoritma önbellek dostudur. Hoca quicksort'un bu açıdan iyi bir seçenek olduğunu, heap sort'un da öyle olduğunu ve bu ikisinin diğerlerine göre böyle bir artısı bulunduğunu söyledi.

> **Ders sonrası not:** Heap sort'un önbellek dostu olduğu ifadesi tartışmalıdır. Heap sort'ta i indisli düğümün çocukları $2i + 1$ ve $2i + 2$ indislerindedir; büyük dizilerde bu indisler birbirinden uzaklaştığı için ardışık erişim bozulur ve önbellek iyi kullanılmaz. Quicksort ise diziyi iki uçtan ardışık tarar; önbellek açısından karşılaştırmalı algoritmalar arasında öne çıkan odur.

Bu bölümün sonunda hoca karşılaştırmaya dayalı algoritmalarla ilgili öğrenilenleri bağladı: yerinde olma, kararlılık, online ve offline olma, hangi algoritmanın yer değiştirme, hangisinin kaydırma ya da veri kopyalama yaptığı. Ardından ders aradan sonra karşılaştırma yapmayan algoritmalarla devam etti.

7. Sayma Sıralaması (Counting Sort)

Dersin ikinci bölümü, karşılaştırma ile çalışan algoritmaların alternatifi olan ve karmaşıklıkta $n \log n$ düzeyinin altına inen algoritmalara ayrıldı. Hoca genel ilkeyi baştan söyledi: zaman karmaşıklığını azaltabilmek için genelde bellek karmaşıklığından feragat etmek gerekir.

7.1 Temel Fikir ve Ön Koşul

Counting sort, veri setindeki elemanların kaç kez geçtiği bilgisini ve ek bir bellek alanını kullanarak sıralama yapar. Derslerde genelde elemanların benzersiz (unique) olduğu örnekler görüldü, ama her zaman öyle olmak zorunda değil. Bu yaklaşım elemanlar benzersiz de olsa, bir eleman birden fazla tekrar etse de çalışır.

Ama bir özel durum vardır: dizideki en büyük değer kadar yeni bir dizi alanı açmak gerekir. Örnekteki veri seti `4 2 2 8 3 3 1` ve en büyük eleman 8'dir. Bu yüzden 8'e kadar ek alan açılır.

Hoca bunun sınırını bir örnekle gösterdi. Dizi `4 2 2 800 3 3 1` olsaydı 800'e kadar alan açmak gerekirdi. Üstelik 1, 2, 3, 4 kullanıldıktan sonra 800'e kadar açılan gözlerin çoğu sıfır kalır, hiç kullanılmaz. Bu yüzden algoritma, en büyük değeri belirli bir sınırı aşmayan veriler için verimlidir. Bin, on bin, yüz bin gibi büyük değerlerde dezavantajlıdır.

> **Ders sonrası not:** Hoca "8'lik bir ekstra alan" dedi, ama örnekte 0'dan 8'e kadar indisler kullanılıyor. Sayaç dizisinin boyutu $k + 1 = 9$'dur. Aşağıdaki tablolarda da 0'dan 8'e dokuz göz vardır.

7.2 Adım Adım Örnek: `4 2 2 8 3 3 1`

Adım 1: En büyük elemanı bul. Dizi bir kez taranır. En büyük değer $k = 8$.

Adım 2: Sayma. Sayaç dizisi açıldıktan sonra dizi ikinci kez taranır ve her değerin kaç kez geçtiği ilgili göze yazılır. 0'dan hiç yok, 1'den bir, 2'den iki, 3'ten iki, 4'ten bir tane var; 5, 6, 7'den hiç yok; 8'den bir tane var.

Değer	0	1	2	3	4	5	6	7	8
Adet	0	1	2	2	1	0	0	0	1

Adım 3: Kümülatif toplam. Her göze, bir önceki gözün değeri eklenir. Hocanın sözlü hesabı:

- 0'da 0 vardır.
- 1'de: $1 + 0 = 1$.
- 2'de: $2 + 1 = 3$.
- 3'te: $2 + 3 = 5$.
- 4'te: $1 + 5 = 6$.
- 5, 6, 7'de: $0 + 6 = 6$ (ara adım eklendi).
- 8'de: $1 + 6 = 7$ (ara adım eklendi).

Değer	0	1	2	3	4	5	6	7	8
Kümülatif	0	1	3	5	6	6	6	6	7

Bu dizi yardımcı (auxiliary) dizidir. Kümülatif değer, o değere eşit ya da küçük kaç eleman olduğunu söyler; yani o değerin çıktı dizisinde nereye kadar yerleşeceğini gösterir.

Adım 4: Yerleştirme. Hoca "esas hile burada" dedi. Girdi dizisi sırayla dolaşılır. Her eleman için sayaç dizisinde o elemanın gözüne gidilir, oradaki değer bir azaltılır ve eleman, çıktı (output) dizisinde azaltılmış değerin gösterdiği indise yazılır. Sıralı dizi girdi dizisinin kendi üzerinde elde edilemez; bunun için ek bir çıktı dizisi tanımlanır.

Sıra	Eleman	Sayaçtaki değer	Azaltılmış değer	Çıktıda yazıldığı göz
1	4	6	5	5
2	2	3	2	2
3	2	2	1	1
4	8	7	6	6
5	3	5	4	4
6	3	4	3	3
7	1	1	0	0

Sayaç değeri her kullanımda güncellenir. Örneğin ilk 2 yerleştirildiğinde 2'nin gözündeki 3, 2 olur; böylece ikinci 2 geldiğinde bir önceki göze yazılır. Bir değerden birden fazla, hatta n tane bulunabileceği için bu güncelleme gereklidir.

Sonuç: `1 2 2 3 3 4 8`.

> **Ders sonrası not:** Slayttaki örnekte yerleştirme adımı dizinin başından sona doğru yapıldı. Bu durumda eşit elemanların sırası tersine döner: girdideki ilk 2 çıktıda 2. göze, ikinci 2 ise 1. göze yazıldı (yukarıdaki tablonun 2. ve 3. satırı). Aynısı 3'ler için de olur. Yani bu sürüm kararlı değildir. Hocanın kararlı listesine koyduğu counting sort, yerleştirmenin dizinin sonundan başa doğru yapıldığı sürümdür; bu sürümde eşit elemanlar girdideki sırayla yerleşir. Radix sort'un doğru çalışması bu kararlılığa bağlıdır (bkz. Bölüm 8). Bu iki durum python ile denenerek doğrulandı.

7.3 Karmaşıklık

Soru: Counting sort'un zaman karmaşıklığı nedir? Hoca sordu, cevap gelmeyince kendisi açıkladı.

Algoritmada dört döngü vardır:

Döngü	İş	Adım sayısı
1	En büyük elemanı bulma	n
2	Sayma	n
3	Kümülatif toplam	k
4	Yerleştirme	n

Toplam iş $3n+k$ mertebesindedir, yani zaman karmaşıklığı $O(n+k)$'dir. Burada n sıralanacak eleman sayısı, k ise veri setindeki en büyük değerdir. k, açılacak yardımcı dizinin boyutunu belirler ve kümülatif toplam bu dizinin üzerinde yürür. En büyük değer 800 olsaydı 800'lük bir dizi açılır ve kümülatif toplam 800 adım sürerdi.

Bellek karmaşıklığı da açılan yardımcı dizinin boyutuna, yani en büyük değere, dolayısıyla k'ya bağlıdır.

> **Ders sonrası not:** Bellek karmaşıklığına n boyutlu çıktı dizisi de girer. Tam ifade $O(n+k)$'dir: $k+1$ gözlü sayaç dizisi ve n gözlü çıktı dizisi.

Hoca counting sort'un doğrusal zamanlı (linear time) bir sıralama algoritması olduğunu, değerleri $1..k$ aralığında olan veri için çalıştırılabileceğini tekrar hatırlattı. Peki hangi durumda çekiciliğini yitirir? Elemanların değer aralığı $1..n^2$ olursa, $k=n^2$ olur ve karmaşıklık $O(n+n^2)=O(n^2)$'ye çıkar. Bu tercih edilecek bir durum değildir. Yani k'nın büyümesi belli bir noktadan sonra dezavantaj yaratır.

> **Ek bilgi (derste anlatılmadı):** Derste C kodu gösterilmedi. Aşağıdaki kod, derste anlatılan dört döngünün C ile yazımıdır; yerleştirme döngüsü kararlılık için sondan başa yürür. Aynı dosyadaki `radixSortLSD`, Bölüm 8'deki LSD örneğini her basamakta 10 gözlü sayaçla yapar. Kod `gcc -Wall -Wextra` ile uyarısız derlendi; çıktısı `1 2 2 3 3 4 8` ve Bölüm 8'deki üç ara sonuçla aynıdır.

```c
#include <stdio.h>
#include <stdlib.h>

/* Derste anlatilan dort adim: max bul, say, kumulatif topla, yerlestir */
void countingSort(const int A[], int B[], int n)
{
    int i, k = A[0];
    for (i = 1; i < n; i++)          /* 1. en buyuk deger: n adim */
        if (A[i] > k) k = A[i];

    int *C = calloc(k + 1, sizeof(int));   /* 0..k: k+1 goz */
    for (i = 0; i < n; i++)          /* 2. sayma: n adim */
        C[A[i]]++;
    for (i = 1; i <= k; i++)         /* 3. kumulatif toplam: k adim */
        C[i] += C[i - 1];
    for (i = n - 1; i >= 0; i--) {   /* 4. yerlestirme: n adim (sondan basa: kararli) */
        C[A[i]]--;
        B[C[A[i]]] = A[i];
    }
    free(C);
}

/* LSD radix sort: her basamak icin 10 gozlu sayac ile counting sort */
void radixSortLSD(int A[], int n)
{
    int max = A[0], i, exp;
    for (i = 1; i < n; i++) if (A[i] > max) max = A[i];
    int *B = malloc(n * sizeof(int));
    for (exp = 1; max / exp > 0; exp *= 10) {
        int C[10] = {0};
        for (i = 0; i < n; i++) C[(A[i] / exp) % 10]++;
```

```c
        for (i = 1; i < 10; i++) C[i] += C[i - 1];
        for (i = n - 1; i >= 0; i--) B[--C[(A[i] / exp) % 10]] = A[i];
        for (i = 0; i < n; i++) A[i] = B[i];
        printf("exp=%3d: ", exp);
        for (i = 0; i < n; i++) printf("%d ", A[i]);
        printf("\n");
    }
    free(B);
}

int main(void)
{
    int A[] = {4, 2, 2, 8, 3, 3, 1}, n = 7, B[7], i;
    countingSort(A, B, n);
    for (i = 0; i < n; i++) printf("%d ", B[i]);
    printf("\n");

    int R[] = {121, 432, 564, 23, 1, 45, 788};
    radixSortLSD(R, 7);
    return 0;
}
```

Programın çıktısı:

```
1 2 2 3 3 4 8
exp=  1: 121 1 432 23 564 45 788
exp= 10: 1 121 23 432 45 564 788
exp=100: 1 23 45 121 432 564 788
```

8. Taban Sıralaması (Radix Sort)

Radix sort da karşılaştırma yapmadan, ek alan kullanarak çalışır ve kendi içinde counting sort'un mantığını kullanır. Sıralanacak bilginin basamakları (digit) kullanılır; elemanlar adım adım basamaklara göre sıralanır.

8.1 LSD Örneği: `121 432 564 23 1 45 788`

Sayılar en düşük anlamlı basamaktan (least significant digit, LSD) başlanarak sıralanır. Kısa sayıların eksik basamakları 0 kabul edilir (1 = 001, 23 = 023).

Adım	Sıralama ölçütü	Sonuç
Başlangıç		121, 432, 564, 23, 1, 45, 788
1	Birler basamağı	121, 1, 432, 23, 564, 45, 788
2	Onlar basamağı	1, 121, 23, 432, 45, 564, 788
3	Yüzler basamağı	1, 23, 45, 121, 432, 564, 788

Ara adımların okunuşu:

- **Adım 1:** Birler basamakları 1, 2, 4, 3, 1, 5, 8'dir. Birler basamağı 1 olan 121 ve 1 başa gelir; girdide 121 önce olduğu için önce o yazılır. Sonra 432 (2), 23 (3), 564 (4), 45 (5), 788 (8) gelir.
- **Adım 2:** Onlar basamakları sırasıyla 1 için 0, 121 için 2, 432 için 3, 23 için 2, 564 için 6, 45 için 4, 788 için 8'dir. Onlar basamağı 2 olan 121 ve 23, önceki adımdaki sıralarını koruyarak yan yana gelir.
- **Adım 3:** Yüzler basamağına göre sıralanınca bütün sayılar yerine oturur.

Hocanın ifadesiyle, en başta birbirleriyle karşılaştırarak yapılacak yer değiştirme işi burada basamaklara göre yapılmış olur. Bu sonuçlar python ile ve yukarıdaki C koduyla doğrulandı.

Ders sonrası not: Her basamak turunda kullanılan counting sort'un kararlı olması şarttır. Yerleştirme baştan sona yapılan sürümle (Bölüm 7.2) aynı örnek denendiğinde sonuç `45 23 1 121 432 564 788` çıkıyor, yani dizi sıralanmıyor. Hoca da ilerleyen kısımda radix sort'un kararlı olmasının önemine değindi.

8.2 Yardımcı Dizinin Boyutu

Soru: Radix sort içinde counting sort basamak bazında kullanılıyorsa, yardımcı dizinin boyutu ne olmalı?

Counting sort mantığıyla doğrudan yapılsaydı, en büyük değer 788 olduğu için 788'lik bir yardımcı dizi açmak gerekecekti. Ama burada basamak basamak sıralandığı için yaklaşım farklıdır.

Cevap: Bir öğrenci 10 dedi; hoca doğruladı. Onluk sayı sistemi kullanıldığı için her basamak 0'dan 9'a kadar değer alır ve yardımcı dizinin boyutu 10 olmalıdır. İkili sayı sisteminde 2'lik, sekizli sistemde 8'lik bir alan gerekirdi. Yani hangi sayı sistemi kullanılıyorsa o kadar alan açılır. Counting sort ile aynı iş yapılsaydı gereksiz yere çok büyük alan açılmış olacaktı.

8.3 Sabit ve Değişken Uzunluk

Hoca, elemanların sabit uzunluklu (fixed length) mı, değişken uzunluklu (variable length) mı olduğunun ileride önem kazanacağını söyledi. İkinci slayttaki örnekte bütün sayılar üç basamaklıydı; önceki örnekte ise basamak sayıları farklıydı. Üç basamaklı sayılar yine önce birler, sonra onlar, sonra yüzler basamağına göre sıralanarak hizalanır. Bu bilgi, radix sort'un iki türü arasında karar verirken yardımcı olacak.

9. Radix Sort'un Karmaşıklığı ve $n \log n$ Sınırı

Radix sort'un zaman karmaşıklığı $O(d \cdot (n + k))$'dir.

- n: eleman sayısı.
- k: bellekte ne kadarlık alana ihtiyaç olduğunu söyler. Onluk sistem için 10'dur.
- d: basamak (digit) sayısı. Hoca sordu, bir öğrenci basamak sayısı diye cevap verdi.

Her basamak için bir counting sort turu yapılır. Bir tur $O(n + k)$ sürer ve d tur vardır.

Soru: Karşılaştırmaya dayalı sıralamada sınır $n \log n$ idi. $d \cdot (n + k)$ ile bu sınırı indirebildik mi, indirdiysek nereye?

$k = 10$ olduğu biliniyor. Hoca d'nin ne olabileceğini sordu ve cevapladı: çok büyük sayıların sıralandığı varsayılırsa 32, 64, hatta 128 olabilir. Ama sonuçta bu bir sabittir. Örneğin $d = 64$ için karmaşıklık $64 \cdot (n + 10)$ olur. Sabitler atıldığında radix sort'un karmaşıklığı teknik olarak $O(n)$'ye, yani doğrusal (linear) karmaşıklığa iner. Bu, $n \log n$'e göre oldukça iyileştirilmiş bir durumdur.

Ama bunun bir sınırı vardır:

- Az sayıda veri ya da değeri büyük olmayan sayılar için radix sort, quicksort gibi algoritmalarla karşılaştırıldığında performans olarak başarılı sonuç üretmez.
- Basamak sayısı fazla olan örneklerde ise önemli avantaj sağlar. Bu yüzden basamak sayısı arttığında bu tür sıralama yaklaşımları tercih edilir.

Hoca slayttaki bir hesaplamayı da gösterdi: sayının hangi tabanda (base) yazıldığına göre basamak sayısının nasıl hesaplandığı. Slayttaki örnekte onluk tabanda yazılan 104 sayısı 3 basamaklıdır, yani $d = 3$'tür. Bu slaytta karmaşıklık $O(d \cdot (n + b))$ biçiminde yazılmış; k'nın yerine taban b gelmiştir.

10. Dizgilerde (String) Radix Sort

10.1 Basamağın Yerini Ne Alır?

Soru: Şimdiye kadar sayılar konuşuldu. Radix sort'un kullanılabileceği başka bir kümede basamağın yerini ne alabilir?

Cevap: Bir öğrenci "kelimeler" dedi; hoca doğruladı. Kelimelerde, daha genel olarak dizgilerde (string), basamağın yerini karakterler alır. Dizgilerde semboller de bulunabilir.

Soru: Dizgiler radix sort ile sıralanıyorsa k kaç olur?

Cevap:

- Yalnız alfabedeki harfler varsa 26.
- Harf dışında her şey olabilir dersek, en kötü durumda ASCII tablosunun boyutu kadar, yani 256. Bazı karakterler yazdırılamaz (non-printable) olduğu için pratikte o kadar önemli değildir, ama ulaşılabilecek üst sınır bellidir.
- Unicode (UTF) kodlar söz konusuysa 65.536.

d ise en uzun dizginin uzunluğudur. Hoca radix sort'un özellikle uzun dizgilerin, isimlerin sıralanmasında önemli avantaj sağladığını söyledi. Radix sort kararlı bir sıralamadır; bu da önemlidir, çünkü böylece kelimelerin geçiş sırası değişmez.

10.2 LSD Dizgi Sıralaması

Bu slaytlarda farklı bir gösterim kullanıldı: N girdi boyutunu, W ise kelime boyutunu (word size), yani anahtardaki karakter ya da basamak sayısını gösterir. Önceki slayttaki d'nin yerini W almıştır. Karmaşıklık $O(W \cdot N)$ olarak verilir. Slaytta LSD radix sort'un logaritmik sınırın altında performans gösterdiği için umut verici bir çözüm olduğu, bu yüzden büyük tamsayılar ve uzun dizgiler için tercih edildiği belirtiliyor.

Hoca slayttaki kodu döngü döngü anlattı:

- İlk döngü frekansları hesaplar: her karakter için sayaç dizisindeki karşılık gelen göz bir artırılır.
- İkinci döngü kümülatif toplamı hesaplar. Dizgi sıralandığı ve ASCII tablosu 256 karakter içerdiği için yardımcı dizi 256 gözlüdür ($R = 256$). Her göze bir önceki gözün değeri eklenir.
- Üçüncü döngü, sayaç dizisini temel alıp geçici bir dizi kullanarak kayıtları (record) o turdaki karakterlerine göre yeniden düzenler.
- Son döngü, geçici dizideki sonucu asıl diziye geri aktarır; bir sonraki tur bu dizi üzerinden devam eder.

Bütün bunlar bir dış döngünün içindedir. Dış döngüde d indisi $W-1$'den 0'a kadar iner; yani son karakterden başlanır ve her turda bir önceki karaktere geçilir. Sayılarla yapılan işin aynısı dizgiler üzerinde yapılmış olur.

Slayttaki örnekte 11 tane, her biri üç karakterli dizgi vardı:

- Birinci turda son karakterlere göre sıralanır: önce `b`'ler, sonra `d`'ler, sonra `e`'ler.
- İkinci turda ortadaki karaktere göre: `a`'lar, `b`'ler, `c`'ler, `d`'ler, `e`'ler.
- Üçüncü turda ilk karaktere göre sıralanınca bütün kelimeler yerine gelir.

Hoca burada kelimelerin hepsinin eşit, üç karakterlik (sabit) uzunlukta olduğunu tekrar vurguladı; uzunluk değişken olduğunda farklı bir durum ortaya çıkacak.

10.3 Kullanım Alanları

Hoca LSD radix sort'un nerede kullanıldığını şöyle anlattı:

- Büyük veritabanlarında, kullanılan alan değerleri sabit uzunluktaysa: hesap numarası (account number), tarih (date), sosyal güvenlik numarası (social security number) ya da TC kimlik numarası gibi numaralar. Hoca TC kimlik numarasının 11 civarında basamaktan oluştuğunu söyledi. Bu numaraları bütün olarak karşılaştırıp sıralamak yerine basamaklar üzerinden sıralamak daha iyidir. Slaytta insertion sort, merge sort ve quicksort aday algoritmalar olarak gösteriliyor , ama LSD radix sort daha iyi performans gösterdiği için genellikle o tercih edilir.
- Çok sayıda basamaktan oluşan büyük sayılar, örneğin 128 bitlik sayılar. Sayılar büyüdükçe diğer sıralama algoritmalarının performansı belirgin şekilde düşer.

11. LSD ve MSD Radix Sort

11.1 İki Tür

Radix sort'un iki türü vardır:

- **LSD radix sort:** Sağdan sola, en düşük anlamlı basamaktan başlar.
- **MSD radix sort:** En yüksek anlamlı basamaktan başlar.

Hoca sağdan başlanan bir sayı örneği daha gösterdi. Birinci adımın sonunda dizi `531, 42, 74, 437, 8` oldu. Bu dizi birler basamağına göre (1, 2, 4, 7, 8) sıralıdır. İkinci adımda onlar basamağına göre sıralanınca `8, 531, 437, 42, 74` elde edilir. 8'in onlar basamağı 0, 531 ve 437'nin 3, 42'nin 4, 74'ün 7'dir; 531 ve 437 önceki sıralarını korur. Son adımda yüzler basamağına göre sıralanınca dizi `8, 42, 74, 437, 531` olur (son adımın sonucu ara adım olarak eklendi; python ile doğrulandı).

11.2 MSD Radix Sort'un Yaklaşımı

MSD radix sort'ta önce ilk karaktere göre bir gruplama ve sıralama yapılır, sonra ikinci karaktere, sonra üçüncü karaktere göre ilerlenir. Algoritma özyinelemeli (recursive) bir yapıdadır: `a` ile başlayanlar bir grup, `b` ile başlayanlar bir grup, `c` ile başlayanlar bir grup olur. Her grup aynı mantıkla kendi içinde ayrı ayrı sıralanır. Bütün gruplar kendi içinde sıralandığında bütün küme sıralanmış olur.

11.3 MSD'nin Fark Yarattığı Yer: Değişken Uzunluk

Hocanın "bizim için kritik olan" dediği nokta şudur. Anahtarlar sabit uzunlukta değil de değişken uzunluktaysa (variable length key) ne olur? Sondan başlansaydı hangi kelimenin sonundan başlanacaktı? Kelimelerin uzunlukları farklı olduğu için karakterler hizalı değildir. Baştan başlayarak ilerlemek ise yapılan işlem sayısını azaltır:

- Kısa bir kelimenin sonuna gelindiğinde, o kelimeyi artık başka bir ayrıma tabi tutmaya gerek kalmaz. Kalan konumlarda değeri sıfır sayılır.
- Yalnız kalan kelimelerin karakterleri işleme girer. Böylece işlenen karakter sayısı azalır.

  Hocanın vurgusu: Değişken uzunluklu anahtarlar söz konusuysa MSD radix sort mutlaka öne çıkar.

Hoca, sabit uzunluklu dizgilerde de MSD'nin avantajlı olduğu, yapılan işlem sayısının azaldığı örnekler bulunduğunu söyledi; slayttaki bir örnekte bu görülüyordu. Ama bu tamamen kelimelerin ve harflerin dağılımına bağlıdır, olabilir de olmayabilir de. Hoca slaytlardaki ilgili kodları paylaşacağını ve incelenmesini söyledi. Slayttaki bir karşılaştırma, sıralama algoritması seçiminin büyük miktarda veri saklandığında çok daha kritik hâle geldiğini gösteriyordu.

11.4 MSD ile Quicksort Karşılaştırması

	MSD radix sort
Dezavantaj 1	Belleğe rastgele (random) erişildiği için önbellek açısından verimsizdir (cache inefficient).
Dezavantaj 2	Sayaç (counter) için ek alan, ayrıca geçici (temporary) dizi için alan kullanır.
Avantaj	Quicksort'a göre doğrusal (lineer) performans gösterir.

Hoca, MSD radix sort'un basamak sayısı fazla olan örnekler için daha başarılı sonuç ürettiğini söyleyerek konuyu bitirdi.

Örnek Sorular

Bu derste ekrandan alınmış hazır kare ve video yoktur. Aşağıdaki sorular derste sözlü sorulan ve çözülen sorulardır.

Soru 1 (derste sözlü, 00:11:14): Bir veri yapısı için iki çözüm var. Birincisinde yapıyı kurmak kolay ama arama yavaş; ikincisinde kurmak daha uzun sürüyor ama arama hızlı. Hangisi seçilmeli?

> **Çözüm:** Yapının ne sıklıkla oluşturulduğuna bağlıdır. Bir kez kurulup çok kez aranacaksa aramanın hızlı olduğu ikinci çözüm mantıklıdır. Yapı sık sık yeniden kuruluyorsa kurulumun verimli olması gerekir. Problem bunu söylemiyorsa varsayım açıkça yazılmalı ve seçim gerekçelendirilmelidir.

Soru 2 (derste sözlü, 00:22:47): Sıralama algoritmalarını karşılaştırırken zaman ve bellek karmaşıklığı dışında hangi ölçütler kullanılır?

> **Çözüm:** Kararlılık (stable ya da unstable), yerinde sıralama (in-place) ve online ya da offline olma.

Soru 3 (derste sözlü, 00:24:07): Bir sıralama algoritmasının stable olması ne demektir? Sıralanınca `1 2 3 3 4 5 5` olan tahtadaki dizi üzerinden yorumlayın.

> **Çözüm:** Aynı değere sahip elemanlar çıktıda girdideki göreli sıralarını koruyorsa algoritma kararlıdır: girdide önce gelen 3 çıktıda da önce, sonra gelen 3 sonra yer alır; 5'ler için de aynısı geçerlidir. Algoritma bunu garanti etmiyorsa kararsızdır. Kararlılar: bubble, insertion, merge, counting, radix. Kararsızlar: selection, quick, heap.

Soru 4 (derste sözlü, 00:28:42): In-place sıralama algoritması nedir, hangi algoritmalar in-place'tir?

> **Çözüm:** Ek bellek kullanmadan, dizinin kendi üzerinde sıralayan algoritmadır. Bubble, selection, insertion, quick ve heap sort yerindedir; merge sort ve counting sort değildir.

Soru 5 (derste sözlü, 00:31:42): Online sıralama algoritmasına hangi algoritma örnek verilebilir?

> **Çözüm:** Insertion sort. Sıralama sürerken gelen yeni eleman, kaydırma ile olması gereken yere yerleşir ve sıralama bozulmadan devam eder. Diğerleri offline'dır.

Soru 6 (derste sözlü, 00:36:10): Küçük bir veri setiniz var. Hangi sıralama algoritmasını kullanırsınız?

> **Çözüm:** Insertion sort. Kararlıdır, yerinde çalışır ve en iyi durumda $O(n)$'dir.

Soru 7 (derste sözlü, 00:39:21): Cache friendly algoritma ne demektir; bir algoritmanın önbelleği iyi kullanması neden önemlidir?

> **Çözüm:** Ana belleğe erişim zaman alır. Veriyi blok hâlinde L1/L2/L3 önbelleğe alıp oradan işleyen ve bu bloklardan iyi yararlanan algoritma daha hızlı çalışır. Hoca quicksort'u bu açıdan iyi bir seçenek olarak gösterdi (heap sort ile ilgili düzeltme için Bölüm 6.1'e bakın).

Soru 8 (derste sözlü, 01:03:05): Counting sort'un zaman karmaşıklığı nedir?

> **Çözüm:** Dört döngü vardır: en büyüğü bulma (n), sayma (n), kümülatif toplam (k), yerleştirme (n). Zaman karmaşıklığı $O(n + k)$'dir; n eleman sayısı, k en büyük değerdir.

Soru 9 (derste sözlü, 01:07:38): `121 432 564 23 1 45 788` dizisini radix sort ile sıralarken, her basamakta kullanılan yardımcı dizinin boyutu ne olmalıdır?

> **Çözüm:** 10. Onluk sayı sisteminde her basamak 0-9 arası değer alır. Counting sort doğrudan kullanılsaydı en büyük değer 788 kadar alan gerekirdi.

Soru 10 (derste sözlü, 01:10:23): Radix sort'un karmaşıklığı $O(d \cdot (n+k))$ ise d neyi gösterir?

> **Çözüm:** Basamak sayısını.

Soru 11 (derste sözlü, 01:11:05): Radix sort ile karşılaştırmaya dayalı sıralamanın $n \log n$ sınırının altına inilebildi mi?

> **Çözüm:** Evet. $k = 10$, d ise 32, 64 ya da 128 gibi bir sabittir. Örneğin $d = 64$ için iş $64 \cdot (n + 10)$ olur; sabitler atılınca karmaşıklık $O(n)$'dir. Ama az sayıda ya da küçük değerli veride quicksort gibi algoritmalardan iyi değildir.

Soru 12 (derste sözlü, 01:13:39): Sayılar dışında radix sort'un kullanılabileceği bir kümede basamağın yerini ne alır?

> **Çözüm:** Kelimeler (dizgiler); basamağın yerini karakterler alır.

Soru 13 (derste sözlü, 01:14:25): Dizgileri radix sort ile sıralarken k kaç olur?

> **Çözüm:** Yalnız harfler için 26, bütün ASCII karakterleri için 256, Unicode için 65.536 (2^{16}). d ise en uzun dizginin uzunluğudur.

Soru 14 (derste sözlü, 01:25:38): Anahtarlar değişken uzunlukluysa, LSD'deki gibi sondan başlansaydı hangi kelimenin sonundan başlanacaktı?

> **Çözüm:** Kelimeler hizalı olmadığı için sondan başlamanın tutarlı bir anlamı yoktur. MSD radix sort baştan başlar; kısa kelime bittiğinde artık işleme girmez ve işlenen karakter sayısı azalır. Bu yüzden değişken uzunluklu anahtarlarda MSD öne çıkar.

Dersin Toparlaması

Dönemin bu son dersinde sıralama algoritmaları iki aile üzerinden toparlandı. Karşılaştırmaya dayalı algoritmalarda (bubble, selection, insertion, merge, quick, heap) $O(n \log n)$ aşılamayan bir alt sınırdır. Bu algoritmalar yalnız zaman ve bellek karmaşıklığıyla değil, kararlılık, yerinde çalışma ve online olma ölçütleriyle de karşılaştırıldı. Seçim duruma bağlıdır: küçük ya da neredeyse sıralı veride insertion sort, büyük veride quick, merge ya da heap sort, bellek kısıtında heap sort, bağlı listede ve paralel hesaplamada merge sort öne çıkar; kararlılık gerekiyorsa quicksort ve heap sort elenir.

İkinci bölümde $n \log n$ sınırının, ek bellek karşılığında nasıl aşıldığı görüldü. Counting sort değerleri doğrudan sayaç dizisinin indisi olarak kullanır ve $O(n + k)$ sürede sıralar, ama k büyüdükçe verimsizleşir. Radix sort, counting sort'u basamak basamak uygulayarak sayaç dizisini tabanın boyutuna indirir ve $O(d \cdot (n + k))$, sabit d ve k için $O(n)$ süre verir. Dizgilerde basamağın yerini karakterler alır; sabit uzunluklu anahtarlarda LSD, değişken uzunluklu anahtarlarda MSD radix sort tercih edilir. Hoca quicksort ve merge sort'un gelecek dönem Algoritma Analizi dersinde böl ve yönet başlığı altında ayrıntılı işleneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.